



CMT 307: Applied Machine Learning

Coursework 2 Group Project:

Detection and Recognition of Traffic Signs

Group 3 Submission

Student Name	Student No	Email
Alex Miseda Mumbo	25049677	MumboAM@cardiff.ac.uk
Andrew Bell	1500744	BellAJ5@cardiff.ac.uk
Chethan Ramachandra	25075975	RamachandraC@cardiff.ac.uk
Pedram Ebadollahyvahed	25015799	EbadollahyvahedP@cardiff.ac.uk
Sathish Dhamodaran	25012600	DhamodaranS@cardiff.ac.uk
Souleymane Ouedraogo	24095691	OuedraogoS1@cardiff.ac.uk

Supervisor: Yukun Lai

Word Count: 4537

Table of Contents

Table of Contents	ii
1. Introduction and Study Motivation.....	1
2. Literature review / Related work	1
2.1. The Problem: Why Traffic Sign Recognition is Challenging.....	1
2.2. Classical Machine Learning Approaches and Their Limitations.....	1
2.3. Convolutional Neural Networks: Architecture and Motivation	1
2.4. Transfer Learning: ResNet and MobileNetV2.....	2
2.5. Summary.....	2
3. Description of the Dataset and Exploratory Data Analysis (EDA)	2
3.1. Dataset Overview	2
3.2. Class Distribution	3
3.3. Image Size and Quality.....	3
3.4. Within-Class Variation	4
4. Preprocessing	4
5. Implementation.....	5
5.1. Data	5
5.2. Environment selection	5
5.3. Neural Network Architectures	5
5.4. Baseline CNN	6
5.5. Transfer Learning	6
5.6. CNN-Dense Layer Combination	6
5.7. Model Training	6
5.8. Optimisation	6
6. Results	7
6.1. Accuracy and training time	7
6.2. Errors – mis-classification.....	7
6.3. Optimisation.....	8
7. Analysis.....	9
7.1. Model Comparison.....	9
7.2. Comparison with previous work.....	9
7.3. Validation rates	9
7.4. Misidentification	9
7.5. Production testing	10
7.6. Resource considerations	10
8. Conclusion and future work	10
8.1. Future Work	10
References	11
Appendices	A-1

1. Introduction and Study Motivation

This project report presents Group 3's work to 'identify traffic signs and recognise them' (CMT 307 Assessment Proforma, 2026). This machine learning (ML) task (alongside the related problem of sign detection) is essential for autonomous vehicles to operate safely on public roads and is also necessary for modern driver aids in a non-fully-autonomous context, such as adaptive cruise control and in-cockpit sign reminders. Clearly, there is great potential for hazard caused by vehicle behaviour based on the incorrect reading of signs, so a high degree of accuracy is paramount.

The current work is based on a dataset that was first used in a challenge at the International Joint Conference on Neural Networks (IJCNN) 2011, the German Traffic Sign Recognition Benchmark (GTSRB) (Stallkamp et al, 2012). This was a multi-class, single-image classification challenge that has since been used many times both to develop better classification systems and as a mechanism for teaching and demonstration of neural networks, often from a practitioner rather than an academic perspective. (See, for example, PythonGuides.com [18].)

The wide adoption of ML, including neural networks, has been helped by the development of bespoke application libraries, such as Keras ('a deep learning API written in Python' (keras.io)) and PyTorch ('an optimized tensor library for deep learning using GPUs and CPUs' (pytorch.org)), which can be used to implement algorithms without deep coding expertise. For this project, we aimed to investigate the effectiveness of ML using a variety of neural network architectures created in Python; and to use the in-built functionality of these ML libraries to optimise our best model.

2. Literature review / Related work

2.1. The Problem: Why Traffic Sign Recognition is Challenging

Traffic sign recognition (TSR) is a foundational task in Advanced Driver Assistance Systems (ADAS) and autonomous vehicles. The core challenge lies in reliably classifying signs under real-world conditions: motion blur, low lighting, partial occlusion, and extreme variation in image size. The German Traffic Sign Recognition Benchmark (GTSRB), introduced by Stallkamp et al. [1], captures this difficulty with over 50,000 images across 43 classes, images as small as 15×15 pixels, significant class imbalance, and substantial variation in illumination. Stallkamp et al. [1] benchmarked methods from SVMs to neural networks, finding that a multi-column deep neural network achieved 99.46% accuracy, surpassing the human baseline of 98.84%, a landmark result demonstrating that CNNs can outperform humans on structured visual recognition. This project uses GTSRB for recognition, with detection addressed at a high level using the companion German Traffic Sign Detection Benchmark (GTSDB).

2.2. Classical Machine Learning Approaches and Their Limitations

Before deep learning, TSR systems relied on hand-crafted features combined with traditional classifiers. A typical pipeline involved colour-based segmentation via HSV thresholding, followed by shape descriptors such as Histogram of Oriented Gradients (HOG) or local binary patterns, fed into a Support Vector Machine (SVM) or k-Nearest Neighbour classifier. The fundamental limitation is that features are designed by humans rather than learned from data. When images are degraded — as is common in GTSRB, where over 52% of training images fall below a brightness threshold of 80/255 (identified in our Phase 1 analysis) — HOG features become unreliable as gradient information degrades. Stallkamp et al. [1] (2012) showed that the best classical pipeline achieved only around 96% accuracy on GTSRB, below the human baseline, confirming that hand-crafted features are insufficient for robust TSR and motivating the shift to end-to-end deep learning.

2.3. Convolutional Neural Networks: Architecture and Motivation

Convolutional Neural Networks (CNNs) are the dominant architecture for image classification. Shared-weight convolutional filters exploit the spatial structure of images, with deeper layers combining low-level features (edges, colours) into abstract representations (shapes, objects). Batch normalisation [3] stabilises gradient flow and accelerates convergence, while Dropout [30] reduces overfitting, particularly important for GTSRB's minority classes. Our baseline CNN uses three convolutional blocks with batch normalisation, ReLU activation, max pooling, and dropout, achieving 98.23% test accuracy, consistent with Almutairy [12], who reported 98.41% for a comparable lightweight CNN. This confirms that well-regularised shallow CNNs are highly competitive on GTSRB.

2.4. Transfer Learning: ResNet and MobileNetV2

He et al. [14] introduced Residual Networks (ResNet), using skip connections to allow gradients to bypass layers and enabling arbitrarily deep networks without degradation. Transfer learning exploits ImageNet-pretrained weights, which encode general-purpose features such as edges, textures, and shapes, fine-tuned on GTSRB. Idrissi et al. [16] achieved 99.28% with ResNet-50. Our ResNet-18 model, applying differential learning rates and partial freezing, achieved 97.51%, with the gap attributable to fewer parameters and smaller input resolution (48×48 vs. 224×224). For efficiency-critical deployment, Sandler et al. [5] introduced MobileNetV2, which uses inverted residual blocks with depthwise separable convolutions to reduce computation to approximately 300 million FLOPs versus over 1.8 billion for ResNet-18. Almutairy [12] reported MobileNetV2 at 97.27% on GTSRB; our implementation achieved 95.96%, reflecting its trade-off between computational efficiency and peak accuracy on fine-grained class discrimination.

2.5. Summary

The literature consistently establishes that deep CNNs, trained from scratch or fine-tuned from ImageNet, achieve state-of-the-art performance on GTSRB, with leading approaches exceeding 99% accuracy. The choice between architectures reflects a trade-off between accuracy, computational cost, and deployment context. Preprocessing decisions, particularly CLAHE contrast enhancement (addressing GTSRB’s low-brightness bias confirmed in Phase 1) and class-balanced augmentation (addressing the 10.7× imbalance ratio), are consistently identified as critical. For detection, single-stage approaches such as YOLOv8 represent current best practice for real-time inference on GTSDb [13],[19]. This project implements three classification models, Baseline CNN, ResNet-18, and MobileNetV2, spanning the accuracy-efficiency spectrum, combined with a YOLOv8n detection front-end [6], providing a thorough evaluation of the current practical state of the art.

3. Description of the Dataset and Exploratory Data Analysis (EDA)

3.1. Dataset Overview

The dataset used to develop, test and optimise our deep learning models was the GTSRB (German Traffic Sign Recognition Benchmark), sourced from kaggle.com and summarised at Table 1.

Dataset	GTSRB
Task	Classification
Training images (80%)	39,209
Test images (20%)	12,630
Number of classes	43
Image resolution	25×25 to 243×225 px
Image type	Cropped sign only
Annotation	Class label + ROI coordinates
Meta file	Provides shape, colour, and sign type metadata for all 43 classes
Source	kaggle.com [17], based on work by Stallkamp et al. [1]

Table 1: GTSRB Dataset Summary

The 43 classes cover a broad range of German traffic signs including speed limits, prohibitory signs, warning and mandatory signs. GTSRB images are pre-cropped around individual signs and vary significantly in quality, from sharp high-resolution captures to severely pixelated low-light samples. Examples of each class are shown in Appendix A Figure 6. There are a total of 51,839 images pre-split into training and testing sets to ensure a consistent test benchmark. Our analysis focused on understanding the data characteristics that would directly influence preprocessing and model design decisions.

3.2. Class Distribution

Significant Class Imbalance

The dataset exhibits a significant class imbalance. The most represented class (Class 2, Speed Limit 50) contains 2,250 images, while the least represented (Class 0, Speed Limit 20) has only 210, a **10.7× imbalance ratio**. Figure 1 shows this distribution. Several speed limit and priority classes dominate, while warning and informational signs are severely underrepresented. This requires mitigation, such as the use of weighted loss functions, oversampling of minority classes or undersampling of over-represented classes. We decided to use weighted CrossEntropyLoss during training, assigning higher penalties to misclassifications of minority classes.



Figure 1: Number of images per class in the training set, showing 10.7× imbalance

3.3. Image Size and Quality

Small and Variable Image Sizes

Image dimensions vary considerably, with widths ranging from **25 - 243 pixels** (mean 50.8 px) and heights ranging from **25 – 225 pixels** (mean 50.3 px). The distribution is heavily right-skewed, with the majority of widths and heights falling in the 25 – 60 pixel range (Figure 2). This presents a challenge: the smallest images are barely recognisable, while the largest contain clear, sharp sign details.

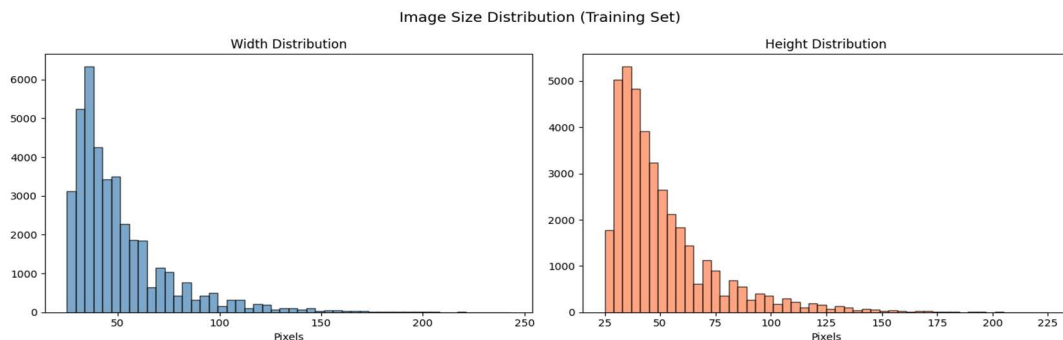


Figure 2: Width and height distributions of training images

However, most images have a square or near-square aspect ratio (mean region of interest (ROI) aspect ratio 1.01 with a standard deviation of 0.08) which means that square resizing will not significantly distort the sign proportions. Therefore, we chose **48×48 pixels** as our resize target, balancing detail preservation with computational efficiency and simplifying the preprocessing pipeline. Although the smallest images will suffer some quality loss due to interpolation, this can be mitigated with other image processing tools and is discussed in the Preprocessing section.

Aspect Ratio

Comparing original images with their ROI-cropped versions (see Appendix A Figure 7) showed minimal difference, as signs already occupy most of the frame. Full images can therefore be used directly for classification, although (in a production scenario) due account would need to be taken for ensuring test images maintained the same aspect ratio.

3.4. Within-Class Variation

Examining multiple samples from the same class (see Appendix Figure 8) revealed substantial quality variation, with a mixture of clearly resolved images and ones that are challenging even for human classification. Within Class 0 alone, images range from sharp 69×70 captures to severely pixelated 27×26 blobs.

Common degradation includes motion blur, low contrast, poor lighting, and colour distortion. This variation motivated our use of Contrast Limited Adaptive Histogram Equalisation (CLAHE) [2] to enhance contrast during preprocessing and **aggressive data augmentation** (rotation, brightness shifts, colour jitter) during training to improve model robustness against real-world image degradation.

4. Preprocessing

Having established some key characteristics of the dataset, including the challenges discussed in the previous section, the preprocessing pipeline was developed using the key findings from the EDA and was further supported by the literature reviewed earlier. Rather than relying on assumptions, each preprocessing step was chosen because the dataset provided clear evidence that it was needed.

Before any transformations were applied, the dataset was checked carefully to ensure it was ready for processing. All 39,209 training images were verified to ensure there were no corrupted files. Brightness was then analysed using 500 randomly selected images, and the results showed that 52.8% had a mean pixel brightness below 80 out of 255, indicating that many images were dark or poorly lit. The dataset's overall mean pixel values [0.342, 0.314, 0.325] were also noticeably lower than the ImageNet averages [0.485, 0.456, 0.406] [8], which was further evidence for the need for contrast enhancement discussed in the EDA.

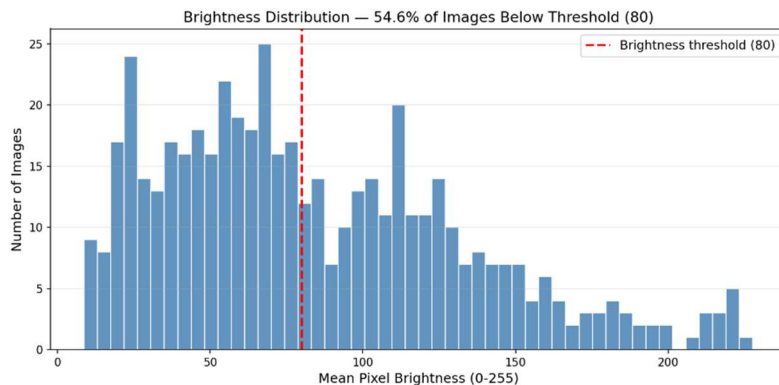


Figure 3: Distribution of Mean Pixel Brightness for 500 Randomly Sampled Images

Resizing

The images were resized to 48×48 pixels using bicubic interpolation (INTER_CUBIC), as this method generally gives better results than bilinear interpolation when upscaling smaller images. This was close to the mean image size (~ 50 x 50 pixels) so matched the natural resolution with minimal quality loss; and kept a square aspect ratio, which again was consistent with the underlying distribution of the raw data.

Contrast Enhancement

The need to enhance contrast resulting from the underlying lack of brightness and quality of the images was met by applying CLAHE contrast enhancement to the L channel in LAB colour space of all the images. Unlike standard histogram equalisation, CLAHE works on smaller local regions of the image and limits contrast enhancement to avoid amplifying noise. This choice was supported by Stallkamp et al. [1] and Sermanet and LeCun [9], who both found contrast enhancement to be beneficial for the GTSRB dataset.

Image normalisation

ImageNet normalisation was then applied by scaling pixel values to the range [0, 1] and standardising them using the ImageNet mean and standard deviation. This step was particularly important for transfer learning with EfficientNet [11], since the pretrained model was originally trained on ImageNet-normalised data, although . He et. al. [4] confirmed this as the standard preprocessing approach for ImageNet-based architectures.

Split out validation data

Although the test images were obtained separately, a set of validation images was split out from the training data to enable an assessment of generalisability to be made during the training of the various models. This split used the stratified train/test split functionality to maintain class proportions, with 80% of the training set images being left as training data and 20% being retained as a validation set. The ability to validate different models against a defined validation set was seen as computationally preferable to using a cross validation approach, and the size of the training set (together with the augmentation approach discussed below) enabled this discrete split.

Augment training data

To improve generalisation and reduce overfitting, training images were augmented with a number of transformations. These included random rotation ($\pm 15^\circ$), affine translations (10%), scale jitter (0.9–1.1 $\times$), colour jitter (brightness, contrast, saturation at 0.3). and shear transformations. These augmentations were based on the work of Ciresan et al. [10], who showed that such transformations improve robustness without changing the meaning of the traffic signs. Horizontal and vertical flipping were deliberately excluded because flipping a traffic sign changes its interpretation entirely.

Address class imbalance

Finally, class imbalance was addressed using a weighted loss function. The EDA revealed a class imbalance ratio of 10.7 to 1 between the most common class (2,250 images) and the least common class (210 images). Following the approach of Johnson and Khoshgoftaar [26] and Kim et al. [25], **weighted Cross Entropy Loss** class weights were calculated in inverse proportion to class frequency, ranging from 0.405 for the most frequent class to 4.342 for the least frequent class. In practice, we found that without class weighting, the model consistently misclassified signs from underrepresented classes during early training epochs. The weighted Cross Entropy Loss helps to ensure that the model does not become biased towards classes with more samples during training.

Tools and Libraries Used

The preprocessing steps were implemented in Python 3.12 using widely available open-source libraries, including Pandas, NumPy, Matplotlib, Seaborn and OpenCV (cv2), with the augmentation processes executed using PyTorch.

5. Implementation

5.1. Data

The GTSRB data is available from various repositories in various formats, but for this project the CSV and png files of training and testing images held at kaggle.com was used. The official test set of 12,630 images was held out entirely and used only for final evaluation. Labels were one-hot encoded for the 43-way softmax output. Initially, the training images were run through the preprocessing pipeline and the validation set extracted. The preprocessed training images were then used as the start point for training various models.

5.2. Environment selection

Although some exploratory model architecture design was carried out using a Keras-Tensorflow environment for simplicity, it proved challenging in Windows to enable such a set up to use laptop GPUs. Therefore, the detailed experimentation and model development was carried out using PyTorch because the group had a competent coding team that was able to deal with the slightly increased complexity of PyTorch and gain the benefit of GPU utilisation for computing efficiency.

5.3. Neural Network Architectures

The project aim was ‘image classification, typically performed using convolutional neural networks’ [14]. Therefore, the experimental focus was on different CNN architectures and approaches. This included the use and performance of pre-trained models, as well as different designs of bespoke CNN. Finally, we attempted to optimise the best performing model from our initial builds using tools such as TensorBoard and the PyTorch-based Ray Tune library.

All models were trained on an NVIDIA RTX 4060 GPU with identical preprocessing, augmentation, and evaluation pipelines. Summaries of each architecture follow below. The code for all pipelines and model training and evaluation was developed in a Jupyter notebook and is viewable as a companion zip file.

5.4. Baseline CNN

The initial CNN was built with three convolutional blocks, each containing two Conv2d, BatchNorm [3], ReLU sequences, followed by 2×2 MaxPooling and 25% spatial dropout. Channels progress from 3→32→64→128. The classifier head flattens the output and passes it through FC(512)→ReLU→Dropout(0.5)→FC(43). This was trained with Adam (learning rate=0.001) and ReduceLROnPlateau scheduling.

5.5. Transfer Learning

Several publicly-available transfer learning models were trialled, varying the number of frozen layers and, in the case of EfficientNet, adjusting the preprocessed image size before training.

ResNet-18 and MobileNetV2

Both these models were fine-tuned from ImageNet pretrained weights. Our initial approach (v1) froze the majority of pretrained layers but obtained poor results. These appear to stem from the large domain gap between ImageNet (224×224 natural images of animals, objects, and scenes) and GTSRB (48×48 symbolic traffic signs), meaning the frozen ImageNet features were simply not transferable to this domain and therefore an amended approach was used.

For the second attempt, we unfroze all residual/inverted-residual blocks and applied **differential learning rates**, 0.0001 for pretrained layers (gentle adaptation) and 0.001 for the new classifier head (fast learning).

EfficientNet B0

This model used EfficientNetB0 pretrained on ImageNet. The idea was to leverage 14 million images worth of learned features and adapt them for traffic sign recognition. We trained in two stages — first freezing the base and training only the classification head (Stage 1, 20 epochs, reaching ~91% validation), then unfreezing the top 30 layers for fine-tuning (Stage 2, 20 more epochs, reaching 97.76% validation accuracy).

5.6. CNN-Dense Layer Combination

After seeing EfficientNet's test performance, we built a custom model from scratch with no pretrained weights. The architecture combined a 5-block CNN feature extractor (32 to 512 filters) with multiple dense layers (512 and 256 neurons) for classification. Building it involved considerable trial and error, starting with fewer layers and gradually increased depth, adjusting the architecture each time the model plateaued. The combination of deep CNN layers with multiple dense classification layers appeared to give a strong performance. The model trained for 30 epochs with Adam optimiser, early stopping, ReduceLROnPlateau, and class weighting. However, the additional model complexity also came with increased computational cost.

5.7. Model Training

We used 25-30 training epochs across our models based on their convergence characteristics and complexity: 25 for the Baseline CNN and transfer learning v1 models; and 30 for the v2 transfer learning models, combined CNN and dense model, and EfficientNet-B0. These values were not arbitrary; all models employed early stopping with a patience of 5 epochs, meaning training terminated automatically if validation accuracy failed to improve for 5 consecutive epochs. In practice, most models triggered early stopping well before reaching their epoch limit. In contrast, The YOLOv8 detection model used 80 epochs with a patience of 15, reflecting the slower convergence typical of object detection tasks on small datasets. Using excessively high epoch counts (e.g., 50 or 100) for classification would not have improved results: as the early stopping mechanism already captures the optimal checkpoint, additional epochs beyond convergence risk overfitting to the training set [29], particularly with our relatively small dataset and the class imbalance [1,2].

5.8. Optimisation

To complement the PyTorch implementation, an independent TensorFlow/Keras implementation of the baseline CNN was developed to provide expanded optimisation monitoring and evaluation. This run used TensorBoard logging to track training and validation accuracy and loss across epochs, providing visual evidence of convergence behaviour and the effect of learning-rate scheduling. The training pipeline also incorporated EarlyStopping, ReduceLROnPlateau and ModelCheckpoint callbacks, ensuring the best-performing checkpoint was used for final evaluation rather than the last epoch.

To systematically evaluate the combined CNN-Dense architecture's sensitivity to its hyperparameters, we conducted a random search across 15 configurations [27], varying learning rate, channel widths, fully connected layer sizes, dropout rates, and weight decay. Each configuration was trained for 10 epochs and evaluated on the validation set. We initially attempted to use Ray Tune with ASHA early stopping [28] for this purpose, but encountered OS-level process management errors on Windows (a known compatibility issue with Ray's multiprocessing on Windows-based Jupyter environments). We therefore implemented an equivalent random search loop directly in PyTorch.

6. Results

The models were evaluated using three measures: overall accuracy in classifying the test images; the confusion matrix generated when classifying the test images (used for comparison of the worst performing image classes); and the model training time (assessed using a common hardware setup for all models).

6.1. Accuracy and training time

Table 2 summarises the numerical results from each model ranked by the overall Accuracy score on the Test dataset. The loss function is used as a gross error check for the accuracy and not used in direct comparison of the models.

Model	Test Acc.	Test Loss	Parameters	Training Time	Notes
Combined CNN-Dense	98.77%	0.0459	Total: 5,912,267 Trainable: 5,912,267	25m 12s	Custom 5-Block CNN + 3-Layer ANN
Baseline CNN	98.07%	0.0657	Total: 2,669,771 Trainable: 2,669,771	22m 32s	Custom 3-block CNN — best performer
ResNet-18 v2	97.99%	0.0998	Total: 11,318,891 Trainable: 11,309,355	37m 38s	Unfrozen + differential LR
MobileNetV2 v2	95.71%	0.2286	Total: 2,562,859 Trainable: 2,552,299	38m 18s	Unfrozen + differential LR
ResNet-18 v1	92.46%	0.3588	Total: 11,318,891 Trainable: 10,635,819	25m 48s	Too many layers frozen
MobileNetV2 v1	83.17%	0.7610	Total: 2,562,859 Trainable: 2,377,835	26m 32s	Too many layers frozen
EfficientNet B0	88.22%	0.3852	Total: 8,058,854 Trainable: 1,835,147	15m 30s	Top 30 Layers Fine-tuned Separate image sizing preprocessing step

Table 2: Summary of Accuracy Results for Different Neural Network Architectures

Despite the significant disparity in trainable parameters, the baseline CNN achieved an accuracy remarkably close to that of the more computationally expensive combined CNN and dense layers. The pre-trained model performances were all less effective than the models trained only on the GTSRB images.

6.2. Errors – mis-classification

For the three best performing models in terms of accuracy, the 5 least accurately classified image classes were established. Due to the class imbalance, this was assessed using an accuracy percentage rather than a count of the absolute number of misclassified images. These results are shown in Figure 4.

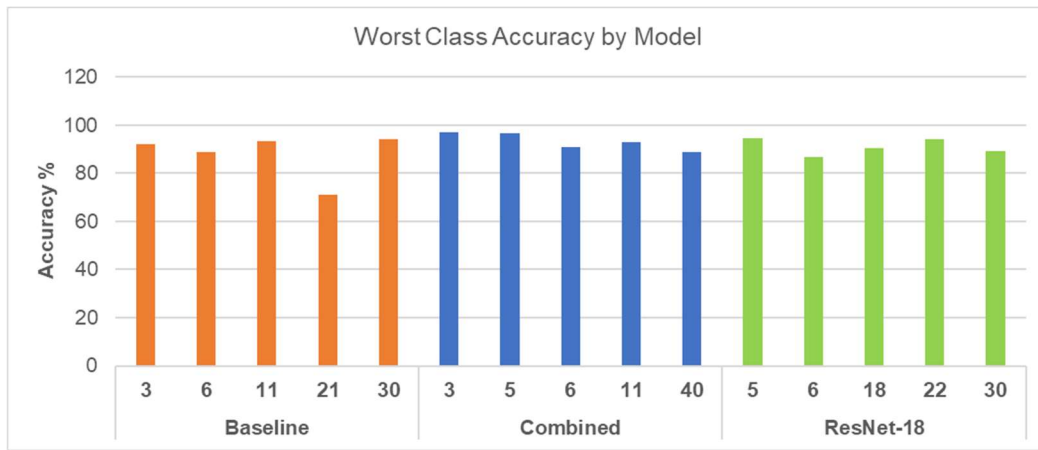


Figure 4: Worst class accuracy for five worst classes in selected models

Additionally, the confusion matrix for each model was used to identify patterns of consistent misclassification, ie when the same (wrong) class was predicted for a given image class. These results, taken from manual interrogation of the confusion matrices for the same model/class combinations as above (at Appendix A Figures 10-12), are shown in Table 3 below.

Model	Actual	Predicted	Frequency
Combined	11	30	26
	3	5	10
	5	2	9
	6	2	5
	40	38	5
Baseline	3	5	11
	6	42	4
	11	30	23
	21	30	2
	30	11, 21	2
ResNet-18 (v2)	5	2	12
	6	20, 30	1
	18	26	22
	22	25	12
	30	11	3

Table 3: Consistent misclassifications for selected models

6.3. Optimisation

Following optimisation work, the best tuned configuration of the CNN-Dense model achieved 99.12% validation accuracy, compared to 98.98% validation accuracy for the manual configuration (note: the reported 98.77% for CNN+ANN elsewhere in this report refers to test set accuracy, which is evaluated on a separate held-out set and is not directly comparable to validation accuracy).

The tuned model favoured a smaller, less regularised architecture: fewer channels in blocks 4–5 (128 and 256 vs 256 and 512), a narrower classifier head (512→256→256 vs 1024→512→256), and consistently lower dropout rates (0.15/0.30/0.25/0.15 vs 0.25/0.50/0.40/0.30). This suggests the original manual design was slightly over-parameterised, the model benefits from fewer parameters with less aggressive regularisation, allowing it to learn more freely from the training data. The results also highlight the instability of deeper configurations: trials with high dropout (0.35 spatial) or very high learning rates (0.005) frequently collapsed to below 60% accuracy, indicating that this architecture is sensitive to the regularisation–capacity balance.

7. Analysis

7.1. Model Comparison

The effectiveness of the project's own CNNs is a legitimate result: even with many fewer trainable parameters, the restriction of training to a very specific data set ensured better adaptation than the deeper, more general-purpose ResNet (~11.2M parameters), for example. The v1 vs v2 comparison further demonstrates that transfer learning is not automatic — it requires careful tuning of what to freeze and how to set learning rates, particularly when the domain gap is large.

The most surprising outcome was that our custom model significantly outperformed EfficientNet, as the group had expected pre-trained models to be stronger. The likely reason it did not work as well was domain mismatch: EfficientNet's ImageNet features were designed for diverse everyday images rather than a highly ordered set of standardised traffic signs. The custom CNN, trained entirely on traffic sign data, learned features that were directly relevant to the problem. The additional dense layers in the classification head further improved its ability to distinguish between the 43 classes.

7.2. Comparison with previous work

The gap between our ResNet-18 (97.51%) and the literature's ResNet-50 (99.28%) is attributable to two factors: first, ResNet-18 has fewer parameters and less representational capacity; second, literature results often use larger input resolutions (224×224) whereas our images are resized to 48×48 in the preprocessing pipeline to better reflect the training image set. This is an acknowledged limitation — rescaling to 224×224 would be expected to improve performance at the cost of computational overhead.

7.3. Validation rates

During model training, there were several occasions when the validation accuracy would exceed the training accuracy through a series of epochs. An example of this can be seen in the model results graph in Appendix A Figure 9. This is likely to be a result of the data augmentation that was applied to the training data after the validation set had been split out. Essentially, the validation was done using an easier subset of data so a better accuracy was achieved. This raises a question about the extent to which it would be possible to overtrain on this sort of dataset, where the core images are fixed and finite, and only the image capture varies. This is a very different recognition problem to something like a human face (or an animal species) where the image target is potentially infinitely variable before image optics (eg brightness, contrast, etc) are factored in. This also accords with the finding in S6.3 that a less-regularised model performed better – overtraining (and lack of generalisation) is less of a risk with this classification problem.

7.4. Misidentification

Compared against a human average accuracy of 98.84%, the accuracy results of all three of our best performing models are acceptably close – by definition, many humans would perform much less well than the average value. However, the other key element of output performance was the by-class performance – were some signs consistently mis-identified?

As Table 3 shows, both group-designed CNNs had problems mis-labelling Class 11 as Class 30 and Class 3 as Class 5. Class 6 was in the worst 5 classes of all models, although there was not necessarily a specifically poor failure mode. 5 to 2 was also problematical. These classes are shown in Figure 5.



Figure 5: Pictures of selected image classes.

Looking at what these signs visually represent, the obvious improvement would be to increase the resolution of the images so that the sign shapes can be resolved by the model. This shows that the real world problem has technical hardware aspects as well as model design and testing.

7.5. Production testing

As part of the analysis, the final model was tested against some sign images collected by the project team. This was done to replicate the sort of in-use testing that a potential production model would need.

A test using a UK '30' sign predicted a Yield/Give way output with low confidence. This demonstrated how domain-specific our trained model was – it took some time for humans to identify the differences between UK and German UK signs such as the nuances in shape, font and colour, whereas these completely defeated the ML model. An internet-sourced picture of a German sign was identified without problem. This highlights an important limitation for real world sign recognition – either separate models are needed for every sign set that a vehicle may encounter, or some form of local updating needs to be possible if a vehicle is moved between countries.

7.6. Resource considerations

Although there were variations in the time to train each model, these were less than expected. The key issue was the ability to harness appropriate hardware: non-GPU-based attempts at training the same models ran to hours rather than minutes.

8. Conclusion and future work

This study has shown that even a relatively simple CNN created using open-source libraries and readily available hardware can quickly generate a traffic sign classifier that compares with human cognition. It illustrates some of the potential for CNN ML models in visual tasks.

In practical terms for vehicle-based ML systems, we have also highlighted the likely trade-off in real world applications in terms of overall accuracy on the one hand and 'blind spot' classes that specific models cannot identify with sufficient accuracy.

8.1. Future Work

The current work points to several areas for further development. Firstly, image classification is only one part of a much larger 'visual system' for autonomous vehicles, so development of other parts of that system (sign detection, control response, etc) are clearly essential. Members of the current project team conducted some preliminary study on sign detection discussed in Appendix B. The ability to process a continuous flow of images in real time, rather than a manual sequencing of predictions, is also a key requirement in this field. As we have also noted, the training of ML image classifiers with finite and infinite image types may benefit from different approaches as the problem of overtraining may be different in the two cases.

References

List references in IEEE reference style. See the IEEE Reference Guide for how to cite a variety of references in IEEE style.

- [1] J. S. M. S. J. a. I. C. Stallkamp, "Man vs. computer: Benchmarking machine learning algorithms for traffic sign recognition'," *Neural Networks*, no. 323–332, p. 32, 2012.
- [2] K. Zuiderveld, "Contrast limited adaptive histogram equalization'," San Diego: Academic Press, 1994.
- [3] S. a. S. C. Ioffe, "Batch normalization: Accelerating deep network training by reducing internal covariate shift'," 2015.
- [4] K. Z. X. R. S. a. S. J. He, "Deep residual learning for image recognition'," *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, no. 770–778, 2016.
- [5] M. H. A. Z. M. Z. A. a. C. L.-C. Sandler, "MobileNetV2: Inverted residuals and linear bottlenecks'," *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, no. 4510–4520, 2018.
- [6] G. C. A. a. Q. J. Jocher, "Ultralytics YOLO (Version 8.0)," 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>. [Accessed 2023].
- [7] S. S. J. S. J. S. M. a. I. C. Houben, "Detection of traffic signs in real-world images: The German Traffic Sign Detection Benchmark'," *International Joint Conference on Neural Networks (IJCNN)*, pp. 1–8. *IEEE*, pp. 1-8, 2013.
- [8] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Miami, FL, USA, 2009, pp. 248–255.
- [9] P. Sermanet and Y. LeCun, "Traffic sign recognition with multi-scale convolutional networks," in *Proc. Int. Joint Conf. Neural Networks (IJCNN)*, San Jose, CA, USA, 2011, pp. 2809–2813.
- [10] D. Ciresan, U. Meier, J. Masci, and J. Schmidhuber, "Multi-column deep neural network for traffic sign classification," *Neural Networks*, vol. 32, pp. 333–338, 2012.
- [11] M. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *Proc. 36th Int. Conf. Mach. Learn. (ICML)*, Long Beach, CA, USA, 2019, pp. 6105–6114.
- [12] M. F. Almutairy, "A Lightweight Convolutional Neural Network (CNN) Architecture for Traffic Sign Recognition in Urban Road Networks," *Electronics*, vol. 12, no. 8, p. 1802, 2023. doi: 10.3390/electronics12081802.
- [13] C. Dewi, R.-C. Chen, S.-H. Liu, and X. Yu, "Traffic Sign Detection and Recognition Using YOLO Object Detection Algorithm: A Systematic Review," *Mathematics*, vol. 12, no. 2, p. 297, 2024. doi: 10.3390/math12020297.
- [14] A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 2nd ed. Sebastopol, CA: O'Reilly Media, 2019.
- [15] M. Hussein and W.-X. Zhu, "A real-time ghost machine learning model built on YOLOv8 for traffic road signs detection and classification in Germany," *Multimedia Systems*, vol. 30, p. 344, 2024. doi: 10.1007/s00530-024-01527-1.
- [16] B. N. Idrissi et al., "Traffic Sign Classification using Deep Learning Comparative Study," *Procedia Computer Science*, 2024. doi: 10.1016/j.procs.2024.06.434.
- [17] Kaggle, "GTSRB - German Traffic Sign Detection Benchmark," [Online]. Available: <https://www.kaggle.com/datasets/meowmeowmeowmeowmeow/gtsrb-german-traffic-sign>. [Accessed: May 2026].
- [18] B. Kumar, "Traffic Signs Recognition Using CNN and Keras in Python," *Python Guides*, 2025. [Online]. Available: <https://pythonguides.com/traffic-signs-recognition-using-cnn-and-keras-python/>. [Accessed: May 2026].
- [19] P. Patel, V. Vekariya, J. Shah, and B. Vala, "Detection of traffic sign based on YOLOv8," *AIP Conference Proceedings*, vol. 3107, no. 1, p. 050015, 2024. doi: 10.1063/5.0212701.

- [20] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 39, no. 6, pp. 1137–1149, 2017.
- [21] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Salt Lake City, UT, USA, 2018, pp. 4510–4520.
- [22] K. V. V. Surender et al., "Traffic Sign Detection and Recognition Using YOLO V8 and CNN," in *Proc. ICLST 2024*. Singapore: Springer, 2024. doi: 10.1007/978-981-95-0853-2_14.
- [23] E. Xi, S. Bing, and Y. Jin, "Novel Deep Learning Model for Traffic Sign Detection Using Capsule Networks," *arXiv preprint, arXiv:1805.04424*, 2018.
- [24] J. Zörn, W. Burgard, and A. Valada, "Deep Learning for Large-Scale Traffic-Sign Detection and Recognition," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 1, pp. 7–16, 2021. doi: 10.1109/TITS.2019.2918719.
- [25] Y. Kim, Y. Lee, and M. Jeon, "Imbalanced image classification with complement cross entropy," *Pattern Recognition Letters*, vol. 151, pp. 33–40, 2021. doi: 10.1016/j.patrec.2021.07.017.
- [26] J. M. Johnson and T. M. Khoshgoftaar, "Survey on deep learning with class imbalance," *Journal of Big Data*, vol. 6, no. 1, p. 27, 2019.
- [27] J. Bergstra and Y. Bengio, "Random search for hyper-parameter optimization," *Journal of Machine Learning Research*, vol. 13, pp. 281–305, 2012.
- [28] L. Li, K. Jamieson, G. DeSalvo, A. Rostamizadeh, and A. Talwalkar, "Hyperband: A novel bandit-based approach to hyperparameter optimization," *Journal of Machine Learning Research*, vol. 18, no. 185, pp. 1–52, 2018.
- [29] L. Prechelt, "Early stopping — but when?" in *Neural Networks: Tricks of the Trade*, G. B. Orr and K.-R. Müller, Eds. Berlin: Springer, 1998, pp. 55–69.
- [30] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, no. 1, pp. 1929–1958, 2014.

Appendices

Appendix A: Additional Figures

Training Images

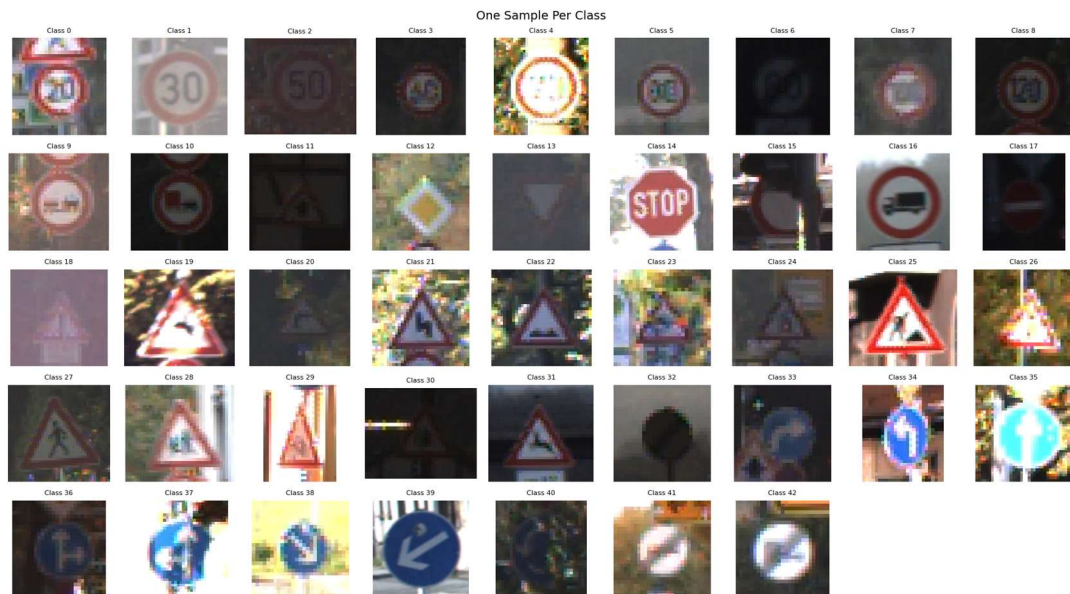


Figure 6: Representative images of all image classes



Figure 7: Original vs ROI-cropped images showing minimal spatial difference

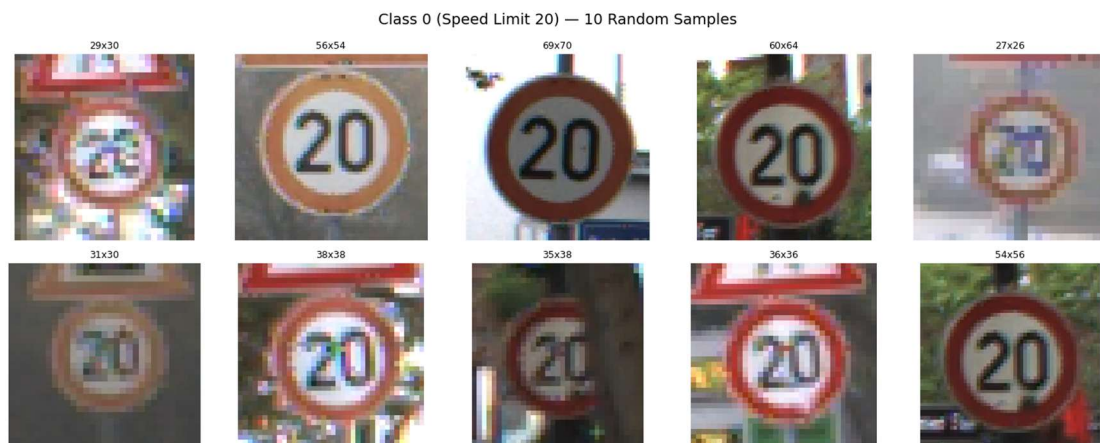


Figure 8: Ten random samples from Class 0 — quality ranges from sharp to heavily degraded

Model performance results

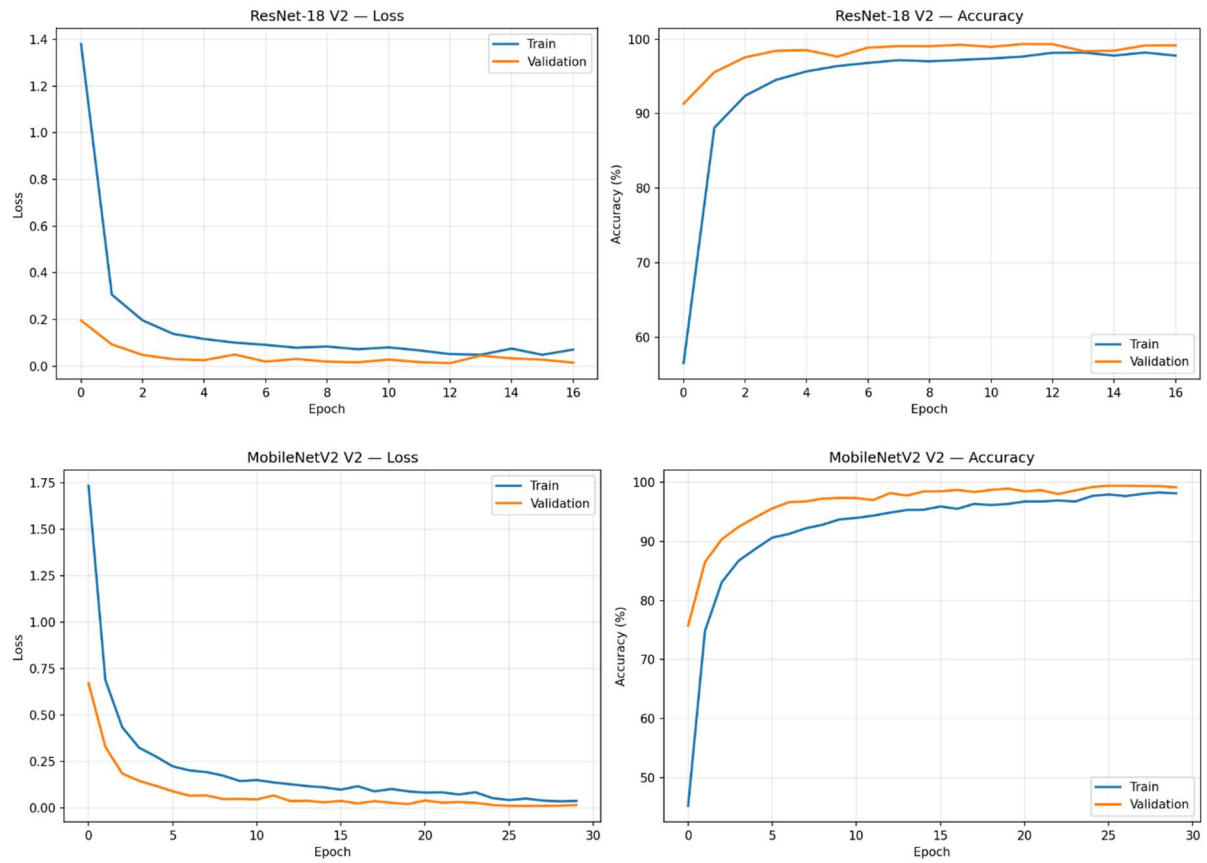
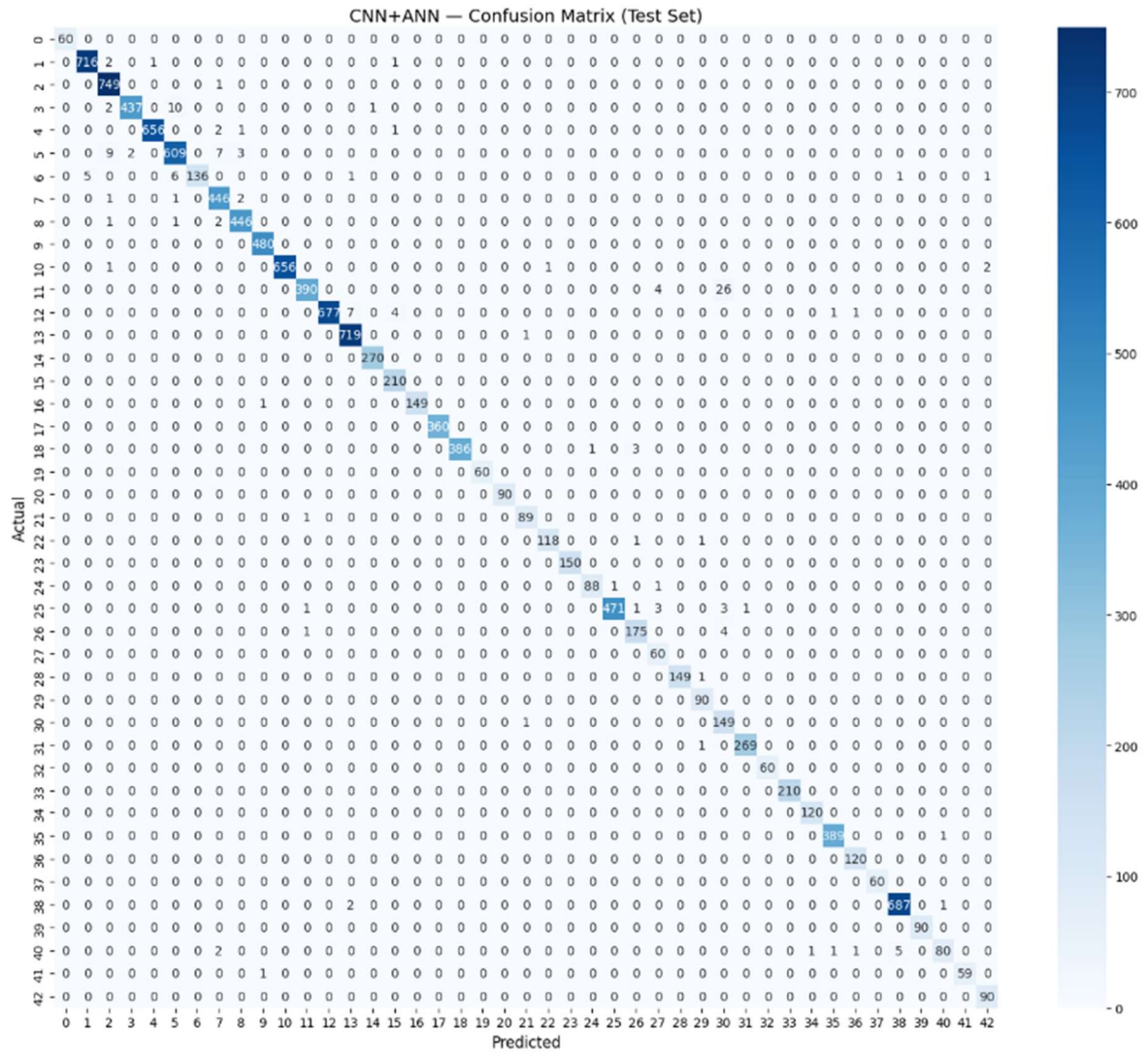
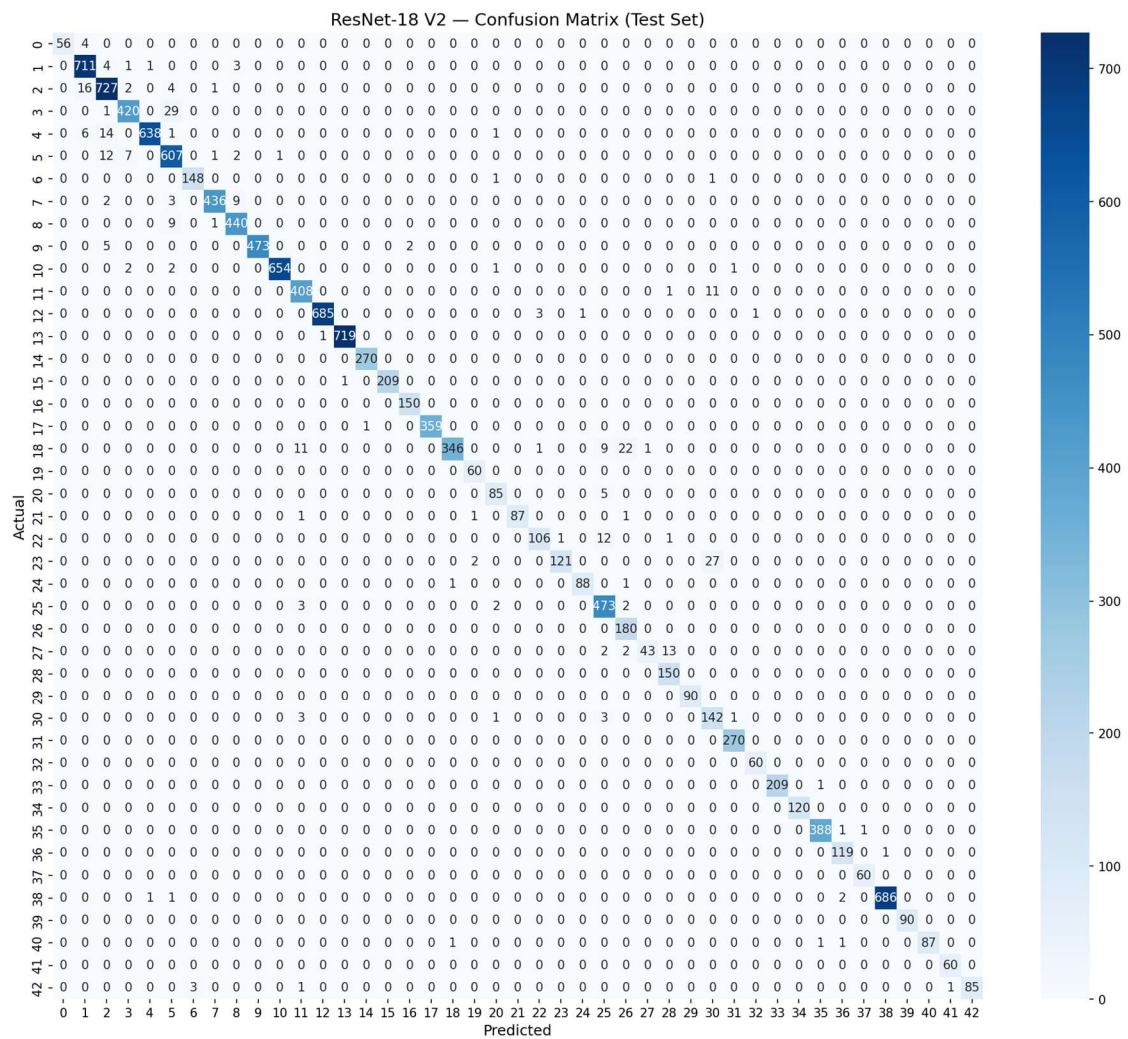


Figure 9: Representative performance curves highlighting validation acc > training acc

Confusion Matrices





Appendix B: Traffic Sign Detection

Object Detection: From Region-Based Methods to YOLO

Recognition (classifying a pre-cropped sign image) and detection (finding signs in a full road-scene image) are distinct problems requiring different architectures. Detection methods must simultaneously answer "where is the sign?" and "what class is it?", which is considerably more difficult.

Two-stage detectors such as Faster R-CNN (Ren et al., 2017) first generate region proposals using a Region Proposal Network (RPN) — a small CNN that slides over the feature map and proposes candidate bounding boxes at multiple scales and aspect ratios. These proposals are then fed into a second stage that classifies each region and refines the box coordinates. This two-stage design achieves high accuracy because the classifier has already been given a candidate region to focus on. Applied to GTSDb, Faster R-CNN Inception ResNet V2 achieved 95.77% accuracy (Dewi et al., 2024). However, the two-stage process introduces latency: even optimised implementations struggle to exceed 10 FPS on embedded hardware, which is insufficient for real-time driving scenarios.

Single-stage detectors such as the YOLO (You Only Look Once) family address this by predicting bounding boxes and class probabilities directly from the full image in a single forward pass. Redmon et al. (2016, cited in Dewi et al., 2024) introduced the original YOLO architecture, which divided the image into a grid and predicted boxes and class scores for each cell. Successive versions (YOLOv2–YOLOv8) progressively improved accuracy through architectural refinements. **YOLOv8** (Ultralytics, 2023) introduces a fully anchor-free detection head and a decoupled architecture that separates classification and localisation into distinct branches, improving gradient flow and reducing the tension between the two tasks that affected earlier versions. Patel et al. (2024) trained YOLOv8 on GTSDb and achieved a mAP@50 of 95.6%, confirming it as the appropriate modern baseline for traffic sign detection. Hussein and Zhu (2024) proposed Ghost-YOLOv8, replacing standard convolutions with computationally cheaper GhostConv modules, achieving 99.4% mAP@50 on a three-class GTSDb setup while reducing compute by 155.2 GFLOPs — demonstrating that YOLOv8 can be further optimised for embedded deployment without significant accuracy loss. A systematic review by Dewi et al. (2024) covering over 100 YOLO-based TSR studies confirms that GTSDb and GTSRB together account for the majority of benchmark evaluations, and that each YOLO iteration has delivered consistent improvements in speed-accuracy trade-off.

Our detection pipeline uses YOLOv8n (the nano variant, prioritising speed), trained in both a 43-class and a single-class configuration. The single-class model simplifies the detection problem to "sign vs. background", with classification delegated entirely to the downstream recognition model — a design choice supported by Surender et al. (2024), who found that pairing a dedicated detector with a specialised classifier outperforms end-to-end single-model approaches for fine-grained recognition.

1.

Detection Approach

For the detection component, we fine-tuned **YOLOv8 (nano variant)** [6] on the GTSDb dataset containing 600 training and 300 test full-scene dashcam images (1360×800 pixels) [7]. Our initial attempt trained YOLO to detect and classify all 43 sign types simultaneously, which produced poor results (**5.77% mAP@50**) due to extreme data scarcity, and some classes had only 1–2 training examples across the entire dataset.

We resolved this by reformulating detection as a **single-class problem**: the detector only needs to answer "is there a traffic sign here?" without identifying its type. This dramatically improved performance to **91.42% mAP@50** with 87.57% precision and 83.66% recall (Table 2). Classification is then handled by cropping detected regions and passing them to our Baseline CNN (98.23% accuracy).

Approach	mAP@50	Precision	Recall	Classes
43-class detection	5.77%	20.27%	11.24%	43
Single-class detection	91.42%	87.57%	83.66%	1

Table 4: YOLOv8 detection results on GTSDb test set (300 images)

End-to-End Pipeline

The final system uses a two-stage architecture: YOLOv8 scans the dashcam image and outputs bounding boxes around detected signs. Each detection is cropped (with 10% padding), enhanced with CLAHE, resized to 48×48, and classified by the Baseline CNN into one of 43 categories. Figure 9 shows the pipeline in action on test images, with both the detected bounding boxes and the predicted class labels displayed.



Figure 13: End-to-end pipeline results — YOLOv8 detection followed by CNN classification